

개인 맞춤형 혈당관리 플랫폼 소프트웨어 및 RAG 기반 AI 상담 챗봇 소스코드

소스코드 (발체) — 혈당관리 플랫폼 S/W · RAG 기반 AI 상담 챗봇

등록 대상	2건 — ① 개인 맞춤형 혈당관리 플랫폼 소프트웨어(B2B용) ② 개인 혈당데이터 분석용 RAG 기반 AI 상담 챗봇
① 플랫폼 S/W	연속혈당측정(CGM) 기기 연동 및 혈당 자료 수집 · 혈당 지표(TIR/TAR/TBR/CV/GMI) 산출과 도표 표시 · 식사·운동 기록과 혈당 연관 분석 · 회원 인증 및 개인 의료정보 관리 · PWA 설치형 모바일 화면
② RAG 챗봇	RAG(검색증강생성) 구조 — ①질의로 혈당 지식베이스를 검색(Retrieval)해 근거가 있으면 그 내용으로 응답 ②없으면 이용자의 혈당 분석 결과(유형 판정·연령별 관리목표·주의 음식·추천 운동)를 프롬프트에 결합(Augmentation) ③생성형 AI가 상담 문장을 생성(Generation). 검색은 자체 구축 지식베이스의 키워드 가중치 매칭으로 구현해 외부 벡터 DB 없이 동작한다.
개발 언어 및 환경	Java 17 (Spring Framework / 전자정부표준프레임워크) · JSP · JavaScript · MyBatis SQL(MySQL) · HTML5/CSS3 · Apache Tomcat 9 · 생성형 AI API(Google Gemini generateContent)
전체 소스 분량	약 33,800줄 (서드파티 라이브러리 및 KISA 제공 암호모듈 제외)
본 문서의 범위	제1부 = 플랫폼 주요 기능 6개 부분, 제2부 = 챗봇 주요 기능 8개 부분 발체 . AI 챗봇은 플랫폼에 포함되어 동작하며, 제2부에서 모듈 단위로 따로 보인다.

■ 제1부 개인 맞춤형 혈당관리 플랫폼 소프트웨어

① 로그인 세션 검사·만료 처리 [src/main/java/egovframework/sejong/util/SessionCheckInterceptor.java]

```
39  /** 로그인 전(비인증) 접근이 허용되어야 하는 엔드포인트 목록. 로그인/회원가입/약관/자동로그인 흐름 전체. */
40  private static final Set<String> PUBLIC_URIIS = new HashSet<>(Arrays.asList(
41      "/index.do",           // 진입점(세션 유무에 따라 login/main 으로 분기)
42      "/loginPage.do",       // 로그인 화면
43      "/sendSensApi.do",     // 인증문자 발송
44      "/loginCheck.do",      // 인증번호 확인(세션 생성)
45      "/autoLogin.do",       // 자동로그인(세션 생성)
46      "/registerUser.do",    // 회원가입(세션 생성)
47      "/updateUser.do",     // 가입 흐름 중 회원정보 갱신(세션 생성)
48      "/User.do",           // 가입 흐름 중 전화번호 존재 확인
49      "/testUser.do",        // 테스트/간편 로그인(세션 생성)
50      "/testUser2.do",
51      "/testUser3.do",       // 카카오 이메일 로그인(세션 생성)
52      "/callback.do",        // 구글 OAuth 콜백
53      "/getSignList.do",     // 로그인 화면 약관 조회
54      "/alldelete.do",       // 로그인 화면 회원탈퇴
55      "/sampleAjax.do"
56      (... 중략 ...)
57  ));
58  // 공개 URL 은 세션 검사 없이 통과
59  if (PUBLIC_URIIS.contains(path)) {
60      return true;
61  }
62
63  HttpSession session = request.getSession(false);
64  Object user = (session == null) ? null : session.getAttribute("user");
65  if (user != null) {
66      return true; // 로그인 상태 → 통과
67  }
68
69  // 세션 만료/미로그인
70  if (isAjax(request)) {
71      response.setStatus(HttpServletResponse.SC_UNAUTHORIZED); // 401
72      response.setContentType("application/json;charset=UTF-8");
73      response.getWriter().write(
74          "{\"IsSucceed\":false,\"sessionExpired\":true,\""
75          + "\"Message\": \"세션이 만료되었습니다. 다시 로그인해 주세요.\"}");
76      response.getWriter().flush();
77  } else {
78      response.sendRedirect(ctx + "/loginPage.do");
79  }
80  return false;
81  }
```

② 연속혈당(CGM) 자료 수집 — 외부 기기 연동 및 토큰 자동 갱신 (이어서)

```
[src/main/java/egovframework/sejong/blood/web/BloodController.java]
333 @RequestMapping(value = "/getBloodData.do", method = RequestMethod.GET)
334 @ResponseBody
335 public ResponseObject getBloodData(HttpSession session, @RequestParam String start, @RequestParam String end,
336 @RequestParam String accessToken) {
337     ResponseObject response = connection.setRequestProperty("Authorization", "Bearer " + newAcc);
338     responseCode = connection.getResponseCode();
339     System.out.println("Response Code(refresh): " + responseCode);
340 }
341 // [2026-07-11] 서버측 자동 갱신: access token 만료(401/403)면 refresh_token 으로 재발급 후 1회 재시도
342 // Sejong Web 방식 아키텍처 플러그인 Refresh 후크/재로그인 없이 투명하게 갱신
343 InputStream cgmIn = (responseCode <= 200 && responseCode < 300) ? connection.getInputStream() : connection.get
344 tErrorStream();
345 if (responseCode == 401 || responseCode == 403) {
346     BufferedReader refTok = new BufferedReader(new InputStreamReader(cgmIn));
347     String refTok = refTok.readLine();
348     String newAcc = (refTok != null && !refTok.isEmpty()) ? refreshAndSaveToken(user.getUserUid(), refTok) :
349 null;
350 String inputLine;
351 while ((inputLine = in.readLine()) != null) {
352     if (newAcc != null) { json.isSuccess = false; json.Data = "REAUTH"; return json; } // refresh 토큰도 만료
353     response.append(inputLine);
354 }
355 connection = (URLConnection) new URL(tokenUrl + "?" + query).openConnection();
356 in.close();
357 if (responseCode < 200 || responseCode >= 300) { json.isSuccess = false; return json; } // 갱신 후에도 실패
358 System.out.println("Blood Data Response: " + response.toString());
359 String jsonResponse = response.toString();
360 }
```

③ 혈당 통계 산출 — 고·저혈당 발생 시간대 상위 3구간 [src/main/resources/egovframework/sqlmap/mapper/Blood_SQL.xml]

```
98 WITH RECURSIVE hours AS (
99     SELECT 0 AS h
100     UNION ALL
101     SELECT h + 1 FROM hours WHERE h < 23
102 ),
103 win AS (
104     SELECT
105         CONCAT(LPAD(h,2,'0'), ':00~', LPAD(h+1,2,'0'), ':00') AS win, -- 예: 08:00~09:00
106         h AS sort,
107         SUM(CASE WHEN t.UPT_VALUE > 0 AND t.UPT_VALUE < 70 AND HOUR(t.CGM_DTM) = h THEN 1 ELSE 0 END) AS hypo_cn
108         SUM(CASE WHEN t.UPT_VALUE > 180 AND HOUR(t.CGM_DTM) = h THEN 1 ELSE 0 END) AS hyper_c
109 nt
110 FROM hours
111 LEFT JOIN T_BLDINF_TRAN t
112     ON t.USER_UUID = #{userId}
113     AND t.CGM_DTM >= CONCAT("#{start}", ' 00:00:00')
114     AND t.CGM_DTM < DATE_ADD("#{end}", INTERVAL 1 DAY)
115     AND t.UPT_VALUE > 0
116     AND HOUR(t.CGM_DTM) = h
117 GROUP BY h
118 )
119 SELECT
120     IFNULL((
121         SELECT GROUP_CONCAT(win ORDER BY sort ASC SEPARATOR ' , ')
122         FROM (
123             SELECT win, sort, hypo_cnt
124             FROM win
125             WHERE hypo_cnt >= 1
126             ORDER BY hypo_cnt DESC, sort ASC
127             LIMIT 3
128         ) s1
129     )
130     , ''
131 )
```

④ 혈당 변화 화살표 — 직전 측정값 대비 연속 각도 표시 [src/main/webapp/WEB-INF/jsp/main/FAHR_00.jsp]

```
886 ①「화살표 방향 수정」(↗ 인데 옆 숫자는 -2 였다) ②「직관적으로 · 사용자한테 설명해야 하는
887 변수는 배제」 두 지적을 함께 반영한 결과 -
888 · 근거는 **바로 옆 두 수치의 차이(point)** 하나뿐. 분당 변화율(mg/dL/분)·30분 추세처럼
889 화면에 없는 값은 쓰지 않는다. **보고 있는 숫자가 곧 화살표**라 설명이 필요 없다.
890 · 오르면 위(↗↑) · 내리면 아래(↘↓) · 그대로면 수평(→). 색은 숫자와 같은_dirColor.
891 · 기울기는 단계 없이 이어진다 : 0=수평 · ±ARROW_FULL 이상=수직. 이 상수 하나로 조절.
892 ※하단 '현재 혈당 변화' 상태·설명문은 **설명서의 「지난 30분」 기준 그대로**(applyTrend30). */
```

④ 혈당 변화 화살표 — 직전 측정값 대비 연속 각도 표시 (이어서) [src/main/webapp/WEB-INF/jsp/main/FAHR_00.jsp]

```
893 const arrowEl = document.getElementById('bloodArrow');
894 if (arrowEl) {
895     const ARROW_FULL = 10; // 직전 5분에 ±10 mg/dL 이면 수직
896     let _r = _hasBoth ? (point / ARROW_FULL) : 0;
897     if (_r > 1) _r = 1; else if (_r < -1) _r = -1;
898     const _deg = -(_r * 90); // 0=수평(→) · -90=위(↑) · +90=아래(↓)
899     arrowEl.textContent = _hasBoth ? '→' : '='; // 계산 불가는 '=' (수평 화살표는 '안정적'으로 읽힌다)
900     arrowEl.style.display = 'inline-block'; // transform 이 먹으려면 필요(기본 inline 이면 무시된다)
901     arrowEl.style.transition = 'transform .35s ease';
902     arrowEl.style.transform = 'rotate(' + _deg.toFixed(1) + 'deg)';
903     arrowEl.style.color = _hasBoth ? _dirColor : '#6c757d';
904     arrowEl.title = _hasBoth
905         ? (point > 0 ? '5분 전보다 ' + point + ' 높아졌습니다'
906           : point < 0 ? '5분 전보다 ' + Math.abs(point) + ' 낮아졌습니다'
```

⑤ 무활동 1시간 자동 로그아웃 [src/main/webapp/WEB-INF/tiles/main/header.jsp]

```
48 (function(){
49     var IDLE_MIN = 60; // 무활동 허용 시간(분) - web.xml session-timeout 과 같게
50     var LIMIT = IDLE_MIN * 60 * 1000;
51     var timer = null, last = 0, done = false;
52
53     function ctx(){
54         try { return CommonUtil.getContextPath(); } catch(e){ return ''; }
55     }
56     function bye(){
57         if (done) return; done = true;
58         /* 자동로그인을 꺼야 로그아웃이 유효하다 - 안 끄면 로그인 화면에서 곧바로 다시 들어온다
59         (login/main.jsp 의 logout() 주석과 같은 이유). 앱이 아니면 조용히 넘어간다. */
60         try { callAndroid("f102", { phone: (window._userPhone || ''), autoYn:false, saveYn:true }); } catch(e){}
61         alert(IDLE_MIN + '분 동안 사용하지 않아 로그아웃되었습니다.\n다시 로그인해 주세요.');
```

나간다 */

```
62         var go = function(){ location.href = ctx() + '/loginPage.do'; };
63         try { CommonUtil.callAjax(ctx() + '/logout.do', 'POST', '', go); setTimeout(go, 1500); } /* 응답이 없어도
64         catch(e){ go(); }
65     }
66     function reset(){
67         if (done) return;
68         var now = +new Date();
69         if (now - last < 30000) return; // 30초 안에 또 들어온 움직임은 흘린다(마우스·스크롤이 초당 수십 번 온
다)
70         last = now;
71         if (timer) clearTimeout(timer);
72         timer = setTimeout(bye, LIMIT);
73     }
74
75     /* 사람이 화면을 만졌다고 볼 수 있는 것들. capture 로 잡아 화면 어디서 눌러도 걸린다. */
76     ['click','keydown','touchstart','mousedown','mousemove','wheel','scroll','input'].forEach(function(ev){
77         document.addEventListener(ev, reset, true);
78     });
```

⑥ 식사등록 음식 검색 — 터치 기기 대응 목록 닫기 [src/main/webapp/WEB-INF/jsp/main/food/foodMain.jsp]

```
686 function outsideClose(e){
687     if (e.target === inputName) return;
688     if (listbox.contains(e.target)) return;
689     hideList();
690 }
691 document.addEventListener('pointerdown', outsideClose, true);
692 document.addEventListener('touchstart', outsideClose, true);
693 document.addEventListener('mousedown', outsideClose, true); // 마우스 쓰는 PC 화면
694 /* 화면을 굴리면 닫는다 - 목록이 칸에 붙어 있어 굴리면 자리가 어긋난다(휴대폰에서 흔한 동작) */
695 /* 굴리면 닫는다 - **연 직후 0.8초 안의 스크롤은 무시**(키보드가 올라오며 생기는 스크롤) */
```

⑥ 식사등록 음식 검색 — 터치 기기 대응 목록 닫기 (이어서) [src/main/webapp/WEB-INF/jsp/main/food/foodMain.jsp]

```
696 /* ★[2026-08-21] **목록 자체를 굴리는 스크롤은 닫지 않는다** - capture 라 목록 안 스크롤에도
697    이 핸들러가 깨어나, 검색 결과를 굴리려는 순간 목록이 닫혔다(「검색 후 스크롤이 안 됩니다」).
698    페이지(다른 곳) 스크롤은 종전대로 닫는다 - 목록이 칸에서 떨어져 보이는 것을 막는 원래 목적. */
699 window.addEventListener('scroll', function(e){
700     var t = e.target;
701     if (t && t.nodeType === 1 && (t === listBox || listBox.contains(t))) return;    // 목록 안 스크롤
702     if (Date.now() - openedAt < 800) return;
703     hideList();
704 }, true);
705 document.addEventListener('keydown', (e)=>{ if (e.key==='Escape') hideList(); });
```

① 질의 접수 및 검색결과 증강 프롬프트 구성 — Augmentation (서버)

[src/main/java/egovframework/sejong/blood/web/BloodController.java]

```

988 @RequestMapping(value = "/blood/chatAsk.do", method = RequestMethod.POST)
989 @ResponseBody
990 public ResponseObject chatAsk(HttpSession session, @RequestBody HashMap<String, Object> params) {
991     ResponseObject json = new ResponseObject();
992
993     String q = params != null && params.get("q") != null ? params.get("q").toString().trim() : "";
994     if (q.isEmpty()) {
995         json.IsSuccess = false;
996         json.Message = "질문이 비어 있습니다.";
997         return json;
998     }
999     // 프롬프트 인젝션/과금 폭증 방지를 위한 길이 제한
1000    if (q.length() > 500) q = q.substring(0, 500);
1001
1002    if (geminiKey == null || geminiKey.isEmpty() || geminiKey.startsWith("YOUR_")
1003        || geminiUrl == null || geminiUrl.isEmpty()) {
1004        json.IsSuccess = false;
1005        (... 종락 ...)
1013 // == 프롬프트 - 기획 「AI 응답 Sample」(3차 전달자료_AI, 2026-08-13) 반영 ==
1014 // ★수치·유형 판정은 **화면(기존 자료)** 이 하고, LLM 은 **문장만** 만든다.
1015 //   ctx 에 TIR/TAR/TBR/CV 원값과 우리가 판정한 유형이 실려 온다(_chatCtxText).
1016 //   △LLM 에 판정을 맡기면 같은 수치에 다른 유형이 나온다 - 그래서 「다시 계산하지 말라」고 못박는다.
1017 //   ★답변 골격 = 기획안의 3단계 : ㉠현재 상태 → ㉡이유/피드백 → ㉢유지·개선 행동.
1018 //   원본 샘플이 그 구조일 때 "사용자가 즉시 이해하고 행동하기에 가장 효과적"이라고 결론냈다.
1019 String systemInstruction =
1020     "너는 당뇨/혈당 관리를 돕는 친절한 한국어 건강 도우미야. "
1021     + "사용자는 연속혈당측정(CGM)을 사용하는 환자야. "
1022     + "답변은 의학적 진단이 아니라 일반적인 생활관리 참고용이며, 심한 고혈당·저혈당이나 이상 증상은 반드시 담당 의사와
상담하라고 안내해. "
1023     + "답변은 100자 내외로 짧게, 줄바꿈은 <br> 태그로 표기해. "
1024     + "혈당 상태에 대한 질문이면 현재 상태 → 그렇게 보는 이유 → 지금 할 행동 순서로 각각 한 문장씩 써. "
1025     + "번호(㉠)나 머릿기호 없이 자연스러운 문장으로만 써. "
1026     + "아래 참고 정보는 앱이 이미 계산해 화면에 보여준 판정이야. 다시 계산하거나 바꾸지 말고 그대로 근거로 삼아. "
1027     + "숫자·퍼센트·지표 이름(TIR/TAR/TBR/CV)은 답변에 쓰지 마 - 수치는 사용자 화면 표가 이미 보여주고 있어. "
1028     + "판정유형별 방향 - 우수: 현재 식사·운동 유지를 격려 / 고혈당형: 식사량·탄수화물 비율 조절과 식후 걷기 / "
1029     + "저혈당형: 공복 운동 회피, 운동 전후 혈당 확인, 식사 거르지 않기 / 변동형: 식사·운동 시간을 일정하게. "
1030     (... 종락 ...)
1036 try {
1037     String answer = callGemini(systemInstruction, q);
1038     if (answer == null || answer.trim().isEmpty()) {
1039         json.IsSuccess = false;
1040         json.Message = "LLM 응답 없음";
1041         return json;
1042     }
1043     json.IsSuccess = true;
1044     json.Data = answer.trim();
1045     return json;
1046 } catch (Exception e) {
1047     log.error("chatAsk ERROR: " + e.getMessage(), e);
1048     json.IsSuccess = false;
1049     json.Message = "LLM 호출 오류";
1050     return json;

```

② 이용자 연령·당뇨 유형별 관리목표 결합 [src/main/java/egovframework/sejong/blood/web/BloodController.java]

```
1073     egovframework.sejong.util.CgmTarget t = egovframework.sejong.util.CgmTarget.of(age, type);
1074     StringBuilder who = new StringBuilder("이 사용자에게 적용되는 관리 목표는 ");
1075     who.append("TIR ").append(t.tir).append("% 이상, TAR ").append(t.tar).append("% 미만, TBR ").append(t.
tbr).append("% 미만")
1076         .append(" ").append(t.name).append(") 이야. 이 목표를 기준으로 말해. ");
1077     /* ★단정적인 말투를 쓰지 않게 - 이 앱은 진단을 하는 곳이 아니다. */
1078     who.append("상태를 단정하지 말고 '~일 수 있어요', '~ 편이 좋습니다' 처럼 부드럽게 말해. ");
1079     return who.toString();
1080 } catch (Exception e) {
1081     log.warn("userProfileForPrompt 실패(무시하고 진행): {}", e.getMessage());
1082     return "";
1083 }
1084 }
```

③ 연령·당뇨 유형별 혈당관리 목표 기준 [src/main/java/egovframework/sejong/util/CgmTarget.java]

```
37 /** 나이·당뇨 유형에 맞는 기준. 값이 없으면(null) 일반 기준. */
38 public static CgmTarget of(Integer age, String typeName) {
39     if (typeName != null && typeName.startsWith("임신성")) {
40         return new CgmTarget("임신 중 기준", 70, 25, 4,
41             "임신 중에는 목표 범위가 63~140 mg/dL 로 더 좁습니다. 이 화면의 수치는 70~180 기준이라 담당 의사와 함께
확인해 주세요.");
42     }
43     if (age != null && age >= 65) {
44         return new CgmTarget("고령 기준", 50, 50, 1,
45             "나이를 고려해 저혈당을 더 엄격히, 고혈당은 조금 느슨하게 봅니다.");
46     }
47     return new CgmTarget("일반 기준", 70, 25, 4, "");
48 }
```

④ 생성형 AI(Gemini) 호출 — Generation 요청·응답 파싱

[src/main/java/egovframework/sejong/blood/web/BloodController.java]

```
1112 JSONObject genCfg = new JSONObject();
1113 genCfg.addProperty("temperature", 0.4);
1114 // 추론 토큰도 maxOutputTokens 를 함께 소모하므로 넉넉히 잡는다.
1115 // (1024 로 두면 추론에 900+ 를 써버려 답변이 잘림 = finishReason:MAX_TOKENS)
1116 genCfg.addProperty("maxOutputTokens", 2048);
1117 // thinking(추론) 최소화 - 단순 건강 Q&A 라 깊은 추론이 불필요.
1118 // ※ gemini-flash-latest 별칭이 Gemini 3 계열(gemini-3.6-flash)로 바뀌면서
1119 // 기존 thinkingBudget:0 은 400 INVALID_ARGUMENT 로 거부된다(추론 완전 비활성 불가).
1120 // Gemini 3 부터는 thinkingLevel("low"/"high") 을 사용한다.
1121 JSONObject thinkingCfg = new JSONObject();
1122 thinkingCfg.addProperty("thinkingLevel", "low");
1123 genCfg.add("thinkingConfig", thinkingCfg);
1124
1125 JSONObject reqBody = new JSONObject();
1126 reqBody.add("system_instruction", sysInstr);
1127 reqBody.add("contents", contents);
1128 reqBody.add("generationConfig", genCfg);
1129
1130 // API 키는 헤더(x-goog-api-key)로 전달 (URL 쿼리 노출 방지)
1131 URL url = new URL(geminiUrl);
1132 HttpURLConnection conn = (HttpURLConnection) url.openConnection();
1133 conn.setRequestMethod("POST");
1134 conn.setRequestProperty("Content-Type", "application/json; charset=UTF-8");
1135 conn.setRequestProperty("x-goog-api-key", geminiKey);
1136
1137 (... 중략 ...)
1157 // 응답 파싱: candidates[0].content.parts[*].text
1158 // Gemini 3 는 parts 에 추론 전용 조각(text 없이 thoughtSignature 만)이 섞여 올 수 있어
1159 // parts[0] 만 보면 null 이 되므로 text 가 있는 조각을 모두 이어붙인다.
1160 JSONObject root = gson.fromJson(sb.toString(), JSONObject.class);
1161 JSONArray candidates = root.getAsJSONArray("candidates");
1162 if (candidates == null || candidates.size() == 0) return null;
```

④ 생성형 AI(Gemini) 호출 — Generation 요청·응답 파싱 (이어서)

[src/main/java/egovframework/sejong/blood/web/BloodController.java]

```
1163 JSONObject cand = candidates.get(0).getAsJsonObject();
1164 JSONObject candContent = cand.getAsJsonObject("content");
1165 if (candContent == null) return null;
1166 JSONArray parts = candContent.getAsJSONArray("parts");
1167 if (parts == null || parts.size() == 0) return null;
1168 StringBuilder text = new StringBuilder();
1169 for (int i = 0; i < parts.size(); i++) {
1170     JsonElement textEl = parts.get(i).getAsJsonObject().get("text");
1171     if (textEl != null && !textEl.isJsonNull()) text.append(textEl.getString());
}
```

⑤ 질의 처리 흐름 — 검색 적중 시 즉답, 미적중 시 생성형 AI (화면) [src/main/webapp/WEB-INF/jsp/main/Blood_Consult.jsp]

```
1427 function sendChat(){
1428     if (_chatBusy) return; // [2026-08-05] 답변 대기 중 중복 전송 차단
1429     var input = document.getElementById('chatInput');
1430     var q = (input.value || '').trim(); if(!q) return;
1431     input.value = '';
1432     // 직접 입력으로 다른 질문을 하면 질문창 잠금 해제(같은 침을 다시 쓸 수 있게)
1433     if (q !== _lastQuickQ) _lastQuickQ = '';
1434     _addMsg(q, true);
1435
1436     // ④ 로컬 키워드/데이터 즉답 (blood_qa.js 매칭 + 화면 지표 기반)
1437     var local = _chatResponse(q);
1438     if (local != null) {
1439         _chatBusy = true;
1440         setTimeout(function(){
1441             _addMsg(local, false); _chatBusy = false;
1442             /* [2026-08-13] "내 최고혈당 관련 운동, 식사 가이드"처럼 수치+가이드를 함께 물으면
1443             수치 즉답으로 끝내지 않고 LLM 가이드 답변을 이어붙인다(사용자 지적).
1444             화면의 주의음식·추천운동 TOP3 도 참고자료로 함께 넘긴다. */
1445             if (_chatLLMFollow) { _chatLLMFollow = false; _askLLM(q, _chatGuideCtx()); }
1446         }, 280);
1447     }
1448     return;
}
```

⑥ AI 참고자료 생성 및 혈당 유형 4분류 자동 판정 [src/main/webapp/WEB-INF/jsp/main/Blood_Consult.jsp]

```
1783 function _chatCtxText(){
1784     var tir = _metricNum('tir'), tar = _metricNum('tar'), tbr = _metricNum('tbr'), cv = _metricNum('cv');
1785     if (tir == null && tar == null && tbr == null && cv == null) return '';
1786     var parts = [];
1787     if (tir != null) parts.push(tir >= 70 ? '목표범위 유지 양호' : (tir >= 50 ? '목표범위 유지 다소 부족' : '목표범위 유지 낮음'));
1788     if (tar != null) parts.push(tar >= 25 ? '고혈당 시간 김' : (tar >= 10 ? '고혈당 시간 다소 있음' : '고혈당 거의 없음'));
1789     if (tbr != null) parts.push(tbr >= 4 ? '저혈당 반복' : (tbr >= 1 ? '저혈당 가끔' : '저혈당 거의 없음'));
1790     if (cv != null) parts.push(cv > 36 ? '혈당 변동 큼' : '혈당 변동 안정적');
1791     var t = _chatGlucoseType(tir, tar, tbr, cv);
1792     return parts.join(' / ') + (t ? (' / 판정유형 ' + t) : '');
1793 }
1794
1795 /* 혈당 유형 자동 판정 - 기획안 § 「혈당 유형 자동 판정」 4유형.
1796 ★판정은 **우리가 한다**(대한당뇨병학회 CGM 기준: TIR≥70·TAR<25·TBR<4·CV≤36).
1797 LLM 에 맡기면 같은 수치에 다른 유형이 나올 수 있다 - 수치·판정은 기존 자료, 문장만 AI. */
1798 function _chatGlucoseType(tir, tar, tbr, cv){
1799     if (tir == null && tar == null && tbr == null) return '';
1800     if (tbr != null && tbr >= 4) return '저혈당형';
1801     if (tar != null && tar >= 25) return '고혈당형';
1802     if (cv != null && cv > 36) return '변동형';
1803     if (tir != null && tir >= 70) return '우수';
1804     return '변동형';
1805 }
```

⑦ 지식베이스 검색 — Retrieval (키워드 가중치 점수제) [src/main/webapp/WEB-INF/jsp/main/Blood_Consult.jsp]

```
1729 var bestItem = null, bestScore = 0, bestMaxLen = 0, secondScore = 0;
1730 for (var i = 0; i < BLOOD_QA.length; i++) {
1731     var item = BLOOD_QA[i];
1732     if (!item || !item.kw) continue;
1733     var score = 0, maxLen = 0;
1734     for (var j = 0; j < item.kw.length; j++) {
1735         var kw = String(item.kw[j]).toLowerCase();
1736         if (kw && q.indexOf(kw) !== -1) {
1737             score += kw.length;
1738             if (kw.length > maxLen) maxLen = kw.length;
1739         }
1740     }
1741     if (score > bestScore) { secondScore = bestScore; bestItem = item; bestScore = score; bestMaxLen =
maxLen; }
1742     else if (score > secondScore) { secondScore = score; }
1743 }
1744 if (bestItem && bestMaxLen >= 2 && bestScore >= secondScore + 2) {
1745     return bestItem.a;
1746 }
1747 }
```

⑧ 혈당 지식베이스(Knowledge Base) 자료 구조 [src/main/webapp/asset/js/blood_qa.js]

```
1 /* =====
2   혈당 Q&A 지식베이스 (데이터)
3   -----
4   · 새 질문/답변을 추가하려면 아래 BLOOD_QA 배열에 항목만 추가하세요.
5   (소스 로직 수정 불필요 - 이 파일만 편집)
6   · 형식: { kw: ['키워드1', '키워드2', ...], a: '답변 HTML' }
7     - kw : 질문에 이 키워드 중 하나라도 포함되면 매칭 (대소문자 무시)
8     - a  : 표시할 답변 (HTML 허용, 줄바꿈은 <br>)
9   · 위에 있는 항목이 먼저 매칭됩니다. 더 구체적인 항목을 위쪽에 두세요.
10  · 영문 키워드는 소문자로 작성하세요 (예: 'hba1c').
11  · "오늘 평균/최고/최저/추이" 같은 데이터 계산형 답변은 코드(_chatResponse)
12    에서 처리하므로 여기에 넣지 않습니다.
13  ===== */
14 var BLOOD_QA = [
```

※ 각 코드 왼쪽 번호는 해당 파일 내 실제 줄번호이며, 가독성을 위해 발체 구간의 공통 들여쓰기만 제거하였다. 접속정보·인증키가 포함된 설정 파일은 발체 대상에서 제외하였고, 인증키는 소스가 아닌 실행 환경의 환경변수로 주입된다.